×××大学

本科毕业设计（论文）

基于 Spring Boot + Vue 的
宿舍管理系统的设计与实现

学　院：×××学院

专　业：×××专业

班　级：×××班

姓　名：×××

学　号：××××××××

指导教师：×××（职称）

二〇二六年十一月

独创性声明

本人郑重声明：所呈交的毕业设计（论文）是本人在指导教师指导下独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含任何其他个人或集体已经发表或撰写过的研究成果。对本文的研究作出重要贡献的个人和集体，均已在文中以明确方式标明。本人完全意识到本声明的法律结果由本人承担。

作者签名：_____　　日期：　　年　月　日

摘　要

随着高校招生规模的不断扩大，在校学生数量持续增长，学生宿舍作为学生日常生活与学习的重要场所，其管理工作日益繁重。传统的宿舍管理主要依赖纸质登记与电子表格，存在数据分散、更新滞后、查询统计困难、入住分配容易出错等问题，已难以满足高校后勤信息化管理的要求。针对上述问题，本文设计并实现了一套基于 Spring Boot + Vue 的宿舍管理系统。

系统采用前后端分离的 B/S 架构：后端基于 Spring Boot 3 与 Java 21 开发，使用 Spring Data JPA 操作 MySQL 8.0 数据库，通过 Spring Security 结合 JWT 实现无状态登录认证与基于角色的权限控制，并以通用 CRUD 控制器基类统一各业务模块的分页查询与增删改查接口；前端基于 Vue 3、Element Plus 与 ECharts 构建单页应用，通过配置驱动的通用 CRUD 页面组件实现学生、宿舍楼、房间、床位等模块的快速开发，利用 Pinia 管理登录状态，借助 ECharts 在仪表盘中直观展示入住率、各楼栋入住情况等统计数据。系统面向系统管理员、宿舍管理员和学生三类角色，实现了学生管理、宿舍楼管理、房间与床位管理、入住分配与调宿退宿、报修管理、卫生检查、访客登记、水电费管理、请假与晚归审批、通知公告、系统账号管理等功能，并对入住分配过程中的床位占用校验、重复入住校验等关键业务规则进行了严格约束。

经功能测试与实际运行验证，系统运行稳定、操作便捷，各项功能均达到设计要求，能够有效提升宿舍管理工作的效率与规范化水平，具有一定的实用价值与推广意义。

关键词：宿舍管理系统；Spring Boot；Vue；前后端分离；JWT

Abstract

With the continuous expansion of college enrollment, the number of students on campus keeps growing, and dormitory management has become increasingly heavy. Traditional dormitory management relies on paper registration and spreadsheets, which suffers from scattered data, delayed updates, difficult queries and statistics, and error-prone accommodation assignment. To solve these problems, this thesis designs and implements a dormitory management system based on Spring Boot and Vue.

The system adopts a front-end and back-end separated B/S architecture. The back end is developed with Spring Boot 3 and Java 21, uses Spring Data JPA to access a MySQL 8.0 database, and implements stateless authentication and role-based access control with Spring Security and JWT. A generic CRUD controller base class unifies paging, query and CRUD interfaces of all business modules. The front end is a single-page application built with Vue 3, Element Plus and ECharts; a configuration-driven generic CRUD page component accelerates the development of modules such as students, buildings, rooms and beds, Pinia manages the login state, and ECharts visualizes occupancy statistics on the dashboard. Serving administrators, dormitory managers and students, the system provides student management, building management, room and bed management, check-in assignment, bed changing and check-out, repair management, hygiene inspection, visitor registration, utility fee management, leave and late-return approval, notices, and system account management, with strict business rules such as bed-occupancy and duplicate check-in validation.

Functional tests and practical running show that the system is stable and easy to use, meets the design requirements, and can effectively improve the efficiency and standardization of dormitory management.

Key words: dormitory management system; Spring Boot; Vue; front-end and back-end separation; JWT

目　录

摘　要

Abstract

第 1 章　绪论（1.1 研究背景与意义；1.2 国内外研究现状；1.3 研究内容；1.4 论文组织结构）

第 2 章　相关技术介绍（2.1 Spring Boot；2.2 Spring Security 与 JWT；2.3 Spring Data JPA 与 MySQL；2.4 Vue 3 与 Element Plus；2.5 ECharts；2.6 前后端分离与 RESTful 架构）

第 3 章　系统需求分析（3.1 可行性分析；3.2 功能需求分析；3.3 用例分析；3.4 业务流程分析；3.5 非功能需求分析）

第 4 章　系统设计（4.1 系统总体架构设计；4.2 系统功能模块设计；4.3 数据库设计；4.4 接口设计；4.5 安全设计）

第 5 章　系统实现（5.1 开发与运行环境；5.2 后端功能实现；5.3 前端功能实现；5.4 关键业务实现）

第 6 章　系统测试（6.1 测试环境与方法；6.2 功能测试；6.3 测试结果分析）

第 7 章　总结与展望（7.1 总结；7.2 展望）

参考文献

致　谢

附　录（附录 A 数据库表清单；附录 B 演示账号）

注：正式提交时请在 Word 中依据标题样式生成带页码的自动目录，并将封面中的学校、学院、专业、姓名、学号、指导教师等占位信息替换为本人真实信息。

第 1 章　绪论

1.1　研究背景与意义

学生宿舍是高校学生生活、休息与交流的主要场所，宿舍管理水平直接关系到学生的居住体验、校园安全以及学校后勤服务的整体质量。近年来，随着高等教育普及化程度不断提高，多数高校在校生规模已达数千乃至数万人，宿舍楼栋、房间与床位数量庞大，入住调配频繁，加之报修、卫生检查、访客登记、水电收费、请假审批等日常事务种类繁多，宿舍管理工作呈现出数据量大、业务种类多、时效性强等特点。

目前仍有不少学校采用纸质台账加电子表格的传统管理方式：学生入住信息登记在纸质表格中，调宿、退宿靠人工涂改，报修靠电话或口头传达，卫生检查结果以纸质单据逐级上报。这类方式存在明显弊端：第一，数据分散在不同表格与单据中，缺乏统一的数据存储，跨学期、跨楼栋统计汇总十分困难；第二，数据更新滞后，床位实际占用情况与台账记录容易不一致，入住分配时可能出现同一床位重复分配的错误；第三，查询困难，管理人员难以快速获得全校入住率、空床位分布、待处理报修数量等宏观信息，不利于科学决策；第四，大量重复性登记工作占用宿管人员的时间，效率低下且容易出错。

随着信息技术的发展，利用信息化手段改造宿舍管理流程已成为高校后勤改革的必然趋势。开发一套功能完善、操作便捷的宿舍管理系统，将学生、楼栋、房间、床位、入住、报修、卫生、费用等数据统一纳入数据库集中管理，实现入住分配的自动校验、业务流程的线上流转与统计数据的可视化展示，既能够减轻管理人员的工作负担、减少人为错误，又能为学生提供自助查询、在线报修与请假申请等便利服务，对提升高校宿舍管理的信息化、规范化水平具有现实意义。

1.2　国内外研究现状

在国外，高校信息化建设起步较早，宿舍管理通常被纳入校园综合服务平台统一建设，系统功能成熟，注重与学生学籍、财务缴费、门禁系统等其他业务系统的数据互

通，服务流程高度线上化、自助化。其技术路线多采用成熟的企业级框架与云服务，系统稳定性与可扩展性较好。

在国内，随着智慧校园建设的推进，越来越多的高校开始建设或采购宿舍管理系统。早期系统多为基于 C/S 架构的桌面软件，部署维护成本高、跨平台能力差；近年来新建系统普遍采用 B/S 架构与前后端分离模式，后端以 Spring Boot 为代表的 Java 技术栈和基于 SSM（Spring + Spring MVC + MyBatis）的技术栈最为常见，前端则广泛采用 Vue、React 等现代框架。已有研究在入住管理、查寝管理等单项业务上成果较多，但部分系统仍存在功能覆盖不全（缺少水电费、访客、请假审批等模块）、界面体验不佳、安全机制薄弱等问题。综合来看，设计一套功能完整、架构清晰、安全可靠且易于维护扩展的宿舍管理系统，仍有较大的实践价值。

1.3 研究内容

本文围绕高校宿舍管理的实际业务需求，研究并实现一套基于 Spring Boot + Vue 的宿舍管理系统，主要研究内容如下：

（1）需求调研与分析。梳理管理员、宿舍管理员（宿管）、学生三类用户的业务场景，明确学生管理、楼栋管理、房间床位管理、入住分配与退宿、报修、卫生检查、访客登记、水电费、请假审批、通知公告、系统账号等功能需求与非功能需求。

（2）系统架构与数据库设计。采用前后端分离架构完成系统总体设计；依据业务实体及其关系完成数据库概念结构（E-R 图）与逻辑结构设计，共设计 12 张数据表。

（3）后端开发。基于 Spring Boot 3、Spring Data JPA 与 MySQL 实现全部业务接口；使用 Spring Security + JWT 实现登录认证与基于角色的访问控制；通过通用 CRUD 控制器基类复用各模块的分页查询与增删改查逻辑，并针对入住分配、调宿、退宿等核心业务编写带业务规则校验的专用接口。

（4）前端开发。基于 Vue 3、Element Plus、Pinia、Vue Router 与 ECharts 实现单页应用；利用配置驱动的通用 CRUD 页面组件统一各管理模块的列表、搜索、分页与

表单交互；在仪表盘中以统计卡片与图表形式展示入住率等核心指标；为学生角色提供“我的宿舍”自助页面，支持在线报修、请假申请与费用查询。

（5）系统测试。对系统进行功能测试与安全测试，验证登录认证、权限控制、入住分配业务规则等关键功能的正确性，并对测试结果进行分析总结。

1.4 论文组织结构

本文共分为七章。第 1 章绪论，介绍研究背景、意义、国内外研究现状与研究内容；第 2 章介绍系统开发所涉及的相关技术；第 3 章进行系统需求分析，包括可行性、功能需求、用例与业务流程；第 4 章阐述系统总体架构、功能模块、数据库、接口与安全设计；第 5 章说明系统的具体实现；第 6 章给出系统测试方案与结果；第 7 章总结全文并展望后续改进方向。

第 2 章 相关技术介绍

2.1 Spring Boot 框架

Spring Boot 是由 Pivotal 团队基于 Spring 框架推出的快速开发框架，其核心理念是“约定优于配置”：通过起步依赖（Starter）与自动配置机制，开发者只需引入 spring-boot-starter-web 等依赖并编写少量配置，即可快速搭建一个独立运行的 Web 应用。Spring Boot 内嵌 Tomcat 等 Servlet 容器，应用可直接打包为可执行 JAR 运行，极大简化了 Java Web 项目的搭建与部署过程。本系统后端采用 Spring Boot 3.3.5 与 Java 21，使用其 Web、数据访问、安全、参数校验等起步依赖构建。

2.2 Spring Security 与 JWT

Spring Security 是 Spring 生态中功能强大的安全框架，提供认证、授权、会话管理、CSRF 防护等完整的安全能力。本系统将其与 JWT（JSON Web Token）结合实现无状态认证：用户登录成功后，服务器将包含用户名、角色与用户 ID 的声明经密钥签

名后生成 JWT 返回给前端；前端后续请求在 Authorization 请求头中携带该令牌；服务器通过自定义过滤器解析校验令牌并建立安全上下文，再依据接口上的角色注解进行授权判断。相比传统 Session 方式，JWT 无状态的特性更适合前后端分离架构与后续的横向扩展。本系统密码存储采用 BCrypt 哈希算法加盐处理，避免明文或弱哈希带来的泄露风险。

2.3 Spring Data JPA 与 MySQL

JPA（Java Persistence API）是 Java 持久化规范，Hibernate 是其主流实现。Spring Data JPA 在此基础上进一步封装，开发者只需定义继承 JpaRepository 的接口，即可获得常用的增删改查、分页与排序能力，还可通过方法名派生查询与 Specification 实现动态条件查询。本系统使用 Spring Data JPA 完成全部数据访问，并结合实体注解完成对象关系映射。

MySQL 是目前应用最广泛的开源关系型数据库之一，具有性能稳定、生态成熟、易于维护等优点。本系统采用 MySQL 8.0 存储全部业务数据，字符集使用 utf8mb4 以完整支持中文内容。

2.4 Vue 3 与 Element Plus

Vue 是一款渐进式 JavaScript 前端框架，采用组件化开发与响应式数据绑定，能够高效构建用户界面。Vue 3 引入了 Composition API（组合式 API），使逻辑复用与代码组织更加灵活，并带来了更好的 TypeScript 支持与运行性能。本系统前端基于 Vue 3 开发，配合 Vue Router 实现单页应用路由、Pinia 实现登录状态等全局状态管理。Element Plus 是面向 Vue 3 的桌面端组件库，提供表格、表单、对话框、分页、消息提示等丰富的现成组件，能够显著提升管理类系统的开发效率与界面一致性。

2.5 ECharts

ECharts 是百度开源的数据可视化图表库，支持柱状图、折线图、饼图等常用图表类型，配置灵活、交互性强。本系统在仪表盘页面使用 ECharts 绘制各楼栋入住情况柱状图与床位使用/报修状态饼图，帮助管理人员直观掌握宿舍运行状态。

2.6 前后端分离与 RESTful 架构

前后端分离是指前端负责页面渲染与用户交互、后端负责业务逻辑与数据提供，两者通过 HTTP 接口交互的开发模式。该模式职责清晰，允许前后端并行开发与独立部署，也便于同一套后端接口同时服务 Web、移动端等多个客户端。本系统遵循 RESTful 风格设计接口：以资源为中心组织 URL，使用 GET、POST、PUT、DELETE 等 HTTP 方法表达操作语义，以 JSON 作为数据交换格式，并通过统一的返回结构（状态码、提示信息、数据体）与前端约定交互规范。

第 3 章　系统需求分析

3.1 可行性分析

3.1.1 技术可行性

系统后端采用的 Spring Boot、Spring Security、Spring Data JPA 与 MySQL 均为成熟稳定的主流开源技术，前端采用的 Vue 3、Element Plus、ECharts 拥有完善的文档与社区支持；开发团队对上述技术栈较为熟悉，且系统已在实际开发环境中完成搭建与联调验证，因此技术上完全可行。

3.1.2 经济可行性

系统所使用的开发工具、框架与数据库均为开源免费软件，运行仅需一台普通服务器（或云主机）即可承载校内并发访问，无需采购商业软件授权，开发与运维成本低，经济上可行。

3.1.3 操作可行性

系统采用 B/S 架构，用户通过浏览器即可访问，无需安装客户端；界面采用成熟的管理系统布局（左侧菜单 + 右侧工作区），表格、表单、对话框等交互方式符合用户日常使用习惯，并提供演示账号与种子数据便于培训与试用，操作上可行。

3.2 功能需求分析

系统用户分为系统管理员、宿舍管理员（宿管）与学生三类角色。三类角色的权限与主要功能如表 3-1 所示。

表 3-1 系统角色与功能权限

角色	主要权限与功能
系统管理员	拥有全部功能权限；负责系统账号管理（创建/禁用账号、重置密码）、全部基础数据与业务数据的维护、公告发布与统计查看
宿舍管理员	负责日常宿舍业务：学生信息查看、楼栋/房间/床位维护、入住分配与调宿退宿、报修处理、卫生检查登记、访客登记、水电费录入、请假审批、公告发布与统计查看
学生	自助服务：查看本人住宿信息与水电费、在线提交报修、提交请假/晚归申请并查看审批状态、浏览通知公告、维护个人账号密码；学生也可通过自助注册开通账号

从功能模块看，系统需求可归纳为以下十一个方面：（1）学生管理，维护学生学号、姓名、性别、学院、专业、班级、联系方式等档案信息；（2）宿舍楼管理，维护楼栋编号、名称、楼层数、类型、宿管员等信息；（3）房间管理，维护各楼栋下房间的房号、楼层、房型、床位数、设施与状态；（4）床位管理，维护每个房间的床位及

其空闲/已入住/维修状态与入住学生；（5）入住管理，支持入住分配、调宿、退宿，并进行床位占用与重复入住校验；（6）报修管理，学生提交报修、管理员受理并更新处理状态与结果；（7）卫生检查，登记各宿舍的检查日期、各项得分、总分、等级与整改备注；（8）访客登记，记录来访人员信息、被访学生、来访与离开时间；（9）水电费管理，按房间按月录入水电用量与费用金额，记录缴费状态；（10）请假/晚归审批，学生提交申请，管理员审批并记录意见；（11）通知公告与统计，发布置顶公告、系统通知等信息，并在仪表盘汇总学生数、入住率、待处理报修等关键指标。

3.3 用例分析

系统核心用例包括登录、入住分配、调宿、退宿、提交报修、处理报修、登记卫生检查、登记访客、录入水电费、请假申请与审批、发布公告、管理账号等。表 3-2 给出其中最具代表性的“入住分配”用例的详细说明，其余用例与之类似，不再逐一展开。

表 3-2　入住分配用例说明

用例项	说明
用例名称	入住分配
参与者	系统管理员 / 宿舍管理员
前置条件	操作者已登录；被分配学生档案存在且当前不在住；系统中存在空闲床位
基本流程	① 进入入住管理页面，点击“入住分配”；②选择待入住学生；③系统列出全部空闲床位，操作者选择目标床位；④提交后系统校验床位状态与学生入住状态；⑤校验通过，床位置为“已入住”，生成状态为“在住”的入住记录
异常流程	若床位已被占用或学生已在住，系统拒绝分配并给出明确提示，不产生任何数据变更
后置条件	床位与学生绑定，入住率统计即时更新

3.4 业务流程分析

入住分配是本系统的核心业务流程，其流程如图 3-1 所示。宿舍管理员在入住管理页面发起分配，选择学生后系统仅列出空闲床位供选择，提交时后端再次进行双重校验（床位必须为空闲、学生不能存在在住记录），校验通过后在同一事务中完成床位状态更新与入住记录生成，保证数据一致性；任一校验不通过则整体回滚并提示原因。调宿流程与之类似，区别在于需先释放原床位再占用新床位；退宿时则将入住记录置为“已退宿”、记录退宿日期并释放床位。报修流程为：学生提交报修（待处理）→ 管理员受理（处理中）→ 维修完成填写处理结果（已完成）或关闭工单；请假流程为：学生提交申请（待审批）→ 管理员批准或驳回并填写审批意见。

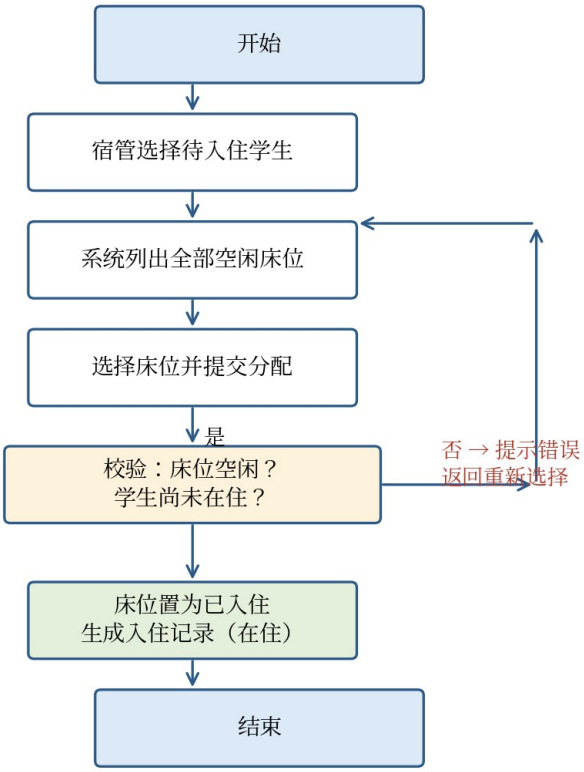


图 3-1　入住分配业务流程图

3.5　非功能需求分析

（1）安全性需求。系统必须对用户身份进行认证，未登录用户不得访问业务接口；不同角色只能访问其权限范围内的功能（如系统账号管理仅管理员可用）；密码必须以强哈希形式存储；接口应统一处理异常，避免向前端泄露内部堆栈信息。

（2）可靠性需求。关键业务操作（入住分配、调宿、退宿）应在数据库事务中执行，确保多张表的数据变更要么全部成功要么全部回滚；删除等危险操作应在前端进行二次确认。

（3）易用性需求。界面布局统一、操作路径简短；列表页面应提供搜索与分页；操作结果应有明确的成功/失败提示；校验失败时应说明具体原因。

（4）性能与可扩展性需求。常规列表查询应在分页机制下快速返回；后端接口按模块划分、前端页面组件化，便于后续新增查寝管理、门禁对接等功能；系统采用前后端分离架构，前后端可独立部署与扩展。

第 4 章　系统设计

4.1　系统总体架构设计

系统采用前后端分离的分层架构，自上而下分为表示层、接口层、业务逻辑层、数据持久层与数据层，如图 4-1 所示。表示层由 Vue 3 单页应用承担，负责页面渲染、交互逻辑与图表展示，通过 Axios 以 JSON 格式调用后端接口；接口层由 Spring MVC 控制器承担，统一进行参数接收、令牌校验与结果封装；业务逻辑层实现各模块的业务规则，核心业务（入住分配等）以事务方式执行；数据持久层由 Spring Data JPA 仓库接口构成，屏蔽底层 SQL 细节；数据层为 MySQL 数据库。分层架构使各层职责单一、耦合度低，便于开发维护与功能扩展。



图 4-1　系统总体架构图

系统的部署结构同样简单清晰：前端代码经 Vite 构建后作为静态资源由后端应用一并对外提供服务（也可独立部署于 Nginx），后端 Spring Boot 应用以可执行 JAR 方式运行并连接 MySQL 数据库，对外仅暴露一个 HTTP 服务端口，便于在校园网服务器上快速部署。

4.2　系统功能模块设计

根据需求分析，系统共划分为十一个功能模块，如图 4-2 与表 4-1 所示。其中基础数据模块（学生、楼栋、房间、床位）是其他业务的数据基础；住宿业务模块（入住管理）是系统的核心；日常事务模块（报修、卫生、访客）与费用审批模块支撑宿舍日常运行；信息与系统模块（公告、仪表盘、账号）提供信息发布、决策支持与系统治理能力。学生角色的自助功能（我的宿舍）复用报修、请假、费用等模块的数据接口，仅做数据范围上的过滤（只能查看和操作本人数据）。

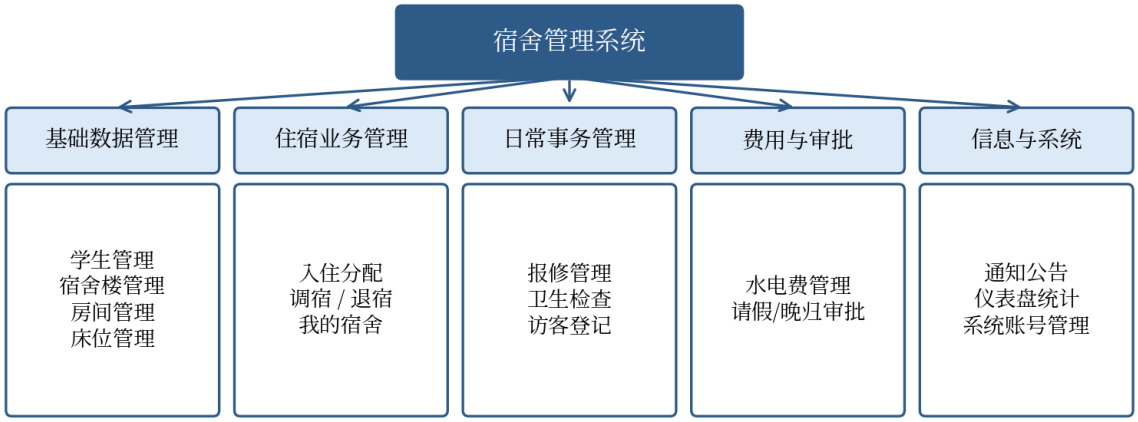


图 4-2　系统功能模块图

表 4-1　系统功能模块一览

模块	主要功能	使用角色
学生管理	学生档案的增删改查、按姓名搜索、分页浏览	管理员、宿管
宿舍楼管理	楼栋信息维护	管理员、宿管
房间管理	房间信息维护、按房号搜索	管理员、宿管
床位管理	床位维护、查看床位占用与入住学生	管理员、宿管
入住管理	入住分配、调宿、退宿、入住记录查询	管理员、宿管
报修管理	报修工单查看、受理、状态流转与结果填写	管理员、宿管
卫生检查	检查记录登记与查询、分数与等级维护	管理员、宿管

访客登记	访客信息登记、来访/离开状态维护	管理员、宿管
水电费管理	按房间按月录入用量与金额、缴费状态维护	管理员、宿管
请假/晚归审批	申请查看、批准/驳回与审批意见填写	管理员、宿管
通知公告	公告发布、置顶与维护	管理员、宿管
仪表盘统计	学生数、入住率、各楼栋入住、报修状态、最新公告展示	全部角色
我的宿舍	查看本人住宿与费用、在线报修、请假申请	学生
系统账号	账号创建、禁用/启用、密码重置	管理员

4.3 数据库设计

4.3.1 概念结构设计

根据业务需求，系统抽象出系统用户、学生、宿舍楼、房间、床位、入住记录、报修工单、卫生检查、访客登记、水电费、请假申请、通知公告共 12 个实体。实体间的主要关系为：一个系统用户账号可与一名学生档案关联（1:0..1）；一名学生可有多条入住记录、报修工单、请假申请与被访记录（1:N）；一栋宿舍楼包含多个房间（1:N）；一个房间包含多个床位（1:N），并可产生多条卫生检查记录与水电费账单（1:N）；一张床位同一时刻至多对应一条在住的入住记录。系统 E-R 图如图 4-3 所示。

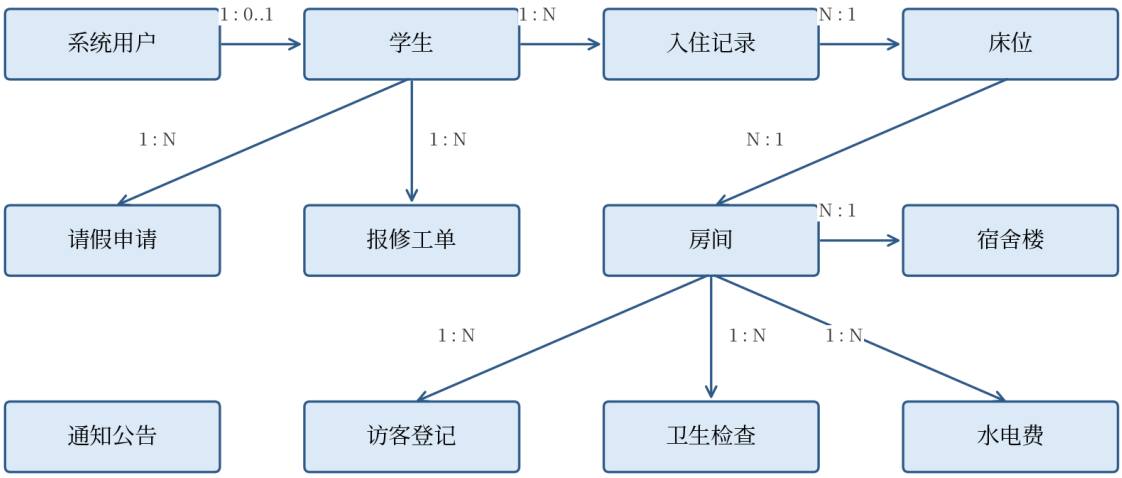


图 4-3　系统 E-R 图

4.3.2　逻辑结构设计

将 E-R 图转换为关系模式，共得到 12 张数据表。各表均以自增整型 id 为主键，实体间关系通过外键字段实现（如 room.building_id 引用 building.id）。主要数据表的结构如表 4-2 至表 4-13 所示（字段类型以 MySQL 实际建表为准，完整建表语句见项目源码 sql/schema.sql）。

表 4-2　sys_user 系统用户表结构

字段名	类型	说明
id	bigint	主键，自增
username	varchar(50)	用户名，唯一
password	varchar(255)	密码（BCrypt 哈希）
real_name	varchar(255)	真实姓名
role	varchar(20)	角色：ADMIN/

		DORM_MANAGER/STUDENT
phone	varchar(255)	联系电话
email	varchar(255)	邮箱
status	int	状态：1 启用，0 禁用
create_time	datetime	创建时间

表 4-3　student 学生表结构

字段名	类型	说明
id	bigint	主键，自增
student_no	varchar(20)	学号，唯一
name	varchar(50)	姓名
gender	varchar(255)	性别
college	varchar(255)	学院
major	varchar(255)	专业
class_name	varchar(255)	班级
grade	varchar(255)	年级
phone	varchar(255)	联系电话
email	varchar(255)	邮箱
id_card	varchar(255)	身份证号
home_address	varchar(255)	家庭住址
status	varchar(255)	状态：在读/毕业/休学
user_id	bigint	关联登录账号 ID
create_time	datetime	创建时间

表 4-4　building 宿舍楼表结构

字段名	类型	说明
id	bigint	主键，自增
code	varchar(20)	楼栋编号，唯一
name	varchar(50)	楼栋名称
floors	int	楼层数
gender_type	varchar(255)	类型：男生宿舍/女生宿舍/混合宿舍
manager_name	varchar(255)	宿管员姓名
manager_phone	varchar(255)	宿管员电话
address	varchar(255)	地址
remark	varchar(255)	备注

表 4-5　room 房间表结构

字段名	类型	说明
id	bigint	主键，自增
building_id	bigint	所属楼栋，外键
room_no	varchar(20)	房号
floor	int	楼层
capacity	int	床位总数
room_type	varchar(255)	房型：四人间/六人间/八人间
gender	varchar(255)	入住性别
facilities	varchar(255)	设施描述
status	varchar(255)	状态：正常/维修中/停用

remark	varchar(255)	备注

表 4-6　bed 床位表结构

字段名	类型	说明
id	bigint	主键，自增
room_id	bigint	所属房间，外键
bed_no	varchar(20)	床号
status	varchar(255)	状态：空闲/已入住/维修
student_id	bigint	入住学生，外键，可空

表 4-7　check_in 入住记录表结构

字段名	类型	说明
id	bigint	主键，自增
student_id	bigint	学生，外键
room_id	bigint	房间，外键
bed_id	bigint	床位，外键
check_in_date	date	入住日期
check_out_date	date	退宿日期，可空
status	varchar(255)	状态：在住/已退宿
remark	varchar(255)	备注（含调宿留痕）
create_time	datetime	创建时间

表 4-8　repair 报修工单表结构

字段名	类型	说明
id	bigint	主键，自增
student_id	bigint	报修学生，外键
room_id	bigint	房间，外键
title	varchar(100)	报修标题
description	varchar(1000)	问题描述
category	varchar(255)	类别：水电/家具/门窗/网络/其他
priority	varchar(255)	优先级：低/普通/高/紧急
status	varchar(255)	状态：待处理/处理中/已完成/已关闭
repairer	varchar(255)	维修人
result	varchar(500)	处理结果
create_time	datetime	报修时间
finish_time	datetime	完成时间

表 4-9　hygiene 卫生检查表结构

字段名	类型	说明
id	bigint	主键，自增
room_id	bigint	房间，外键
inspector	varchar(255)	检查人
check_date	date	检查日期
score	int	总分（0-100）
grade	varchar(255)	等级：优秀/良好/合格/不合格

ground_score	int	地面分
bed_score	int	床铺分
desk_score	int	桌面分
bathroom_score	int	卫生间分
remark	varchar(500)	备注/整改意见

表 4-10　visitor 访客登记表结构

字段名	类型	说明
id	bigint	主键，自增
visitor_name	varchar(50)	访客姓名
visitor_phone	varchar(255)	访客电话
id_card	varchar(255)	访客身份证号
student_id	bigint	被访学生，外键
room_id	bigint	房间，外键
visit_time	datetime	来访时间
leave_time	datetime	离开时间
reason	varchar(300)	来访事由
status	varchar(255)	状态：在访/已离开

表 4-11　fee 水电费表结构

字段名	类型	说明
id	bigint	主键，自增

room_id	bigint	房间，外键
period	varchar(10)	账期，如 2026-10
electricity_usage	double	用电量（度）
water_usage	double	用水量（吨）
electricity_fee	double	电费（元）
water_fee	double	水费（元）
total_amount	double	总金额（元）
status	varchar(255)	状态：未缴费/已缴费
due_date	date	缴费截止日期
pay_time	datetime	缴费时间

表 4-12　dorm_leave 请假申请表结构

字段名	类型	说明
id	bigint	主键，自增
student_id	bigint	申请学生，外键
type	varchar(255)	类型：请假离校/晚归
start_time	datetime	开始时间
end_time	datetime	结束时间
reason	varchar(500)	事由
status	varchar(255)	状态：待审批/已批准/已驳回
approver	varchar(255)	审批人
approve_remark	varchar(300)	审批意见
create_time	datetime	提交时间

表 4-13　notice 通知公告表结构

字段名	类型	说明
id	bigint	主键，自增
title	varchar(100)	标题
content	varchar(5000)	内容
type	varchar(255)	类型：系统公告/宿舍通知/卫生通报/停水停电
publisher	varchar(255)	发布人
top	bit	是否置顶
publish_time	datetime	发布时间

4.4　接口设计

后端共提供 60 余个 RESTful 接口，遵循统一规范：请求与响应均为 JSON 格式；响应体统一包含 code（状态码）、msg（提示信息）、data（数据）三个字段；列表接口统一支持 page、size 分页参数及按字段的模糊查询参数；业务校验失败时返回 code=400 及具体原因。主要接口如表 4-14 所示。

表 4-14　系统主要接口一览

接口	方法	说明
/api/auth/login	POST	用户登录，返回 JWT 与用户信息
/api/auth/register	POST	学生自助注册（创建账号与学生档案）
/api/auth/me	GET	获取当前登录用户及其学生档

		案
/api/auth/change-password	POST	修改密码（校验原密码）
/api/students 等 10 个资源路径	GET/POST/PUT/DELETE	各业务模块的分页查询、全量列表、详情、新增、修改、删除
/api/checkins/assign	POST	入住分配（含床位与重复入住校验）
/api/checkins/{id}/change-bed	POST	调宿（释放旧床位、占用新床位）
/api/checkins/{id}/checkout	POST	退宿（释放床位）
/api/checkins/my/{studentId}	GET	查询学生本人在住信息
/api/dashboard/stats	GET	仪表盘综合统计
/api/users	GET/POST/PUT/DELETE	系统账号管理（仅管理员）
/api/users/{id}/reset-password	POST	重置账号密码为初始密码

系统同时集成 springdoc-openapi，自动生成 OpenAPI 文档并提供 Swagger UI 在线接口调试页面（/swagger-ui），便于开发联调与后续维护。

4.5 安全设计

系统安全设计主要包括：第一，认证安全。采用 JWT 无状态令牌机制，令牌包含用户身份与角色信息并经 HMAC-SHA 密钥签名，有效期 24 小时；密码使用 BCrypt 算法加盐哈希存储，数据库中不保存明文密码。第二，授权控制。基于 Spring Security 过滤器链统一拦截请求，除登录、注册与静态资源外所有接口均需认证；系统账号管理接口通过方法级注解限制为管理员角色，学生越权访问时返回 403。第三，业务安全。入住分配等关键操作在服务端进行二次校验，不依赖前端限制；账号禁用后立即无法登

录。第四，通用防护。全局异常处理器统一封装错误响应，避免泄露内部实现细节；跨域策略通过 CORS 配置统一管理。

第 5 章　系统实现

5.1　开发与运行环境

表 5-1　开发与运行环境

类别	环境
操作系统	Linux（开发服务器）/ Windows、macOS（浏览器访问）
后端语言与框架	Java 21、Spring Boot 3.3.5、Spring Security、Spring Data JPA
前端技术	Vue 3.5、Element Plus 2.8、Pinia、Vue Router、Axios、ECharts 5、Vite 5
数据库	MySQL 8.0
构建工具	Maven 3.8、npm
运行方式	后端打包为可执行 JAR（含前端静态资源），以系统服务方式运行于 8081 端口

系统初始数据由后端的数据初始化组件在首次启动时自动写入，包括管理员与宿管演示账号、12 名学生及其账号、2 栋宿舍楼、24 间房间、96 个床位、10 条入住记录，以及报修、卫生、访客、水电费、请假与公告等演示数据，便于系统演示与验收。

5.2　后端功能实现

5.2.1　统一返回与通用 CRUD 基类

为减少重复代码，后端设计了统一响应类 Result（包含 code、msg、data）与分页响应类 PageResult，并抽象出通用 CRUD 控制器基类 CrudController。该基类基于泛型与 Spring Data JPA 的 Specification 机制，一次性实现了分页查询（可按任意字符串字段模糊匹配）、全量列表、详情、新增、修改与删除六类接口；学生、楼栋、房间、床位、报修、卫生、访客、水电费、请假、公告等十个模块的控制器只需继承该基类并声明实体与仓库类型即可获得完整接口，单个模块的控制器代码仅约 10 行。基类同时预留 beforeSave 校验钩子，便于个别模块扩展自身的保存前校验逻辑。

5.2.2 登录认证与权限控制

认证流程如下：前端提交用户名与密码至 /api/auth/login，后端查询用户并使用 BCrypt 校验密码，同时检查账号是否被禁用；校验通过后生成包含用户名、角色、用户 ID 且带有 24 小时有效期的 JWT 返回前端。前端将令牌保存在 localStorage 中，并在 Axios 请求拦截器中自动为每个请求添加 Authorization 请求头；后端的 JWT 过滤器解析令牌、恢复用户身份与角色并写入安全上下文；Spring Security 依据“除登录、注册与静态资源外均需认证”的规则及方法级角色注解完成授权判断。学生访问管理员专属接口时将得到 403 响应，前端据此提示无权访问。

5.2.3 仪表盘统计

仪表盘统计接口聚合多个仓库的查询结果，一次性返回学生总数、楼栋与房间数、床位总数/已占用/空闲数、入住率（保留一位小数）、在住人数、各状态报修数量、未缴费账单数、卫生检查平均分、各楼栋床位占用明细以及最新 5 条公告，前端据此渲染统计卡片与图表，避免多次请求造成的数据不一致。

5.3 前端功能实现

5.3.1 配置驱动的通用 CRUD 页面

前端管理类页面具有高度一致的交互模式：搜索框 + 数据表格 + 分页 + 新增/编辑对话框。为避免为每个模块重复编写页面，系统设计了通用组件 CrudPage 与模块配置

表 modules.js：每个模块只需以配置形式声明接口路径、表格列、表单字段（文本、数字、下拉、日期、外键选择、开关、长文本等类型）与行级操作，通用组件即自动完成数据的分页加载、关键字搜索、外键选项的远程加载、表单的组装与提交（外键字段自动组装为 {id: …} 形式）、删除二次确认等全部逻辑。入住管理模块还通过配置注入了“退宿”行级操作与专用的“入住分配”对话框（学生与空闲床位两个下拉选择）。该设计使新增一个管理模块只需增加一段配置，显著降低了开发与维护成本。

5.3.2 仪表盘与学生自助页面

仪表盘页面在加载时请求统计接口，将核心指标以卡片形式展示，并使用 ECharts 绘制各楼栋入住/空闲床位对比柱状图与床位使用、报修状态构成饼图，下方以时间线形式展示最新公告。学生登录后菜单自动切换为学生视图，“我的宿舍”页面汇总展示本人住宿信息（楼栋、房号、床号、入住日期）、本宿舍水电费、本人报修记录，并提供在线报修与请假/晚归申请表单，提交后数据即时进入管理员的处理队列。

5.4 关键业务实现

入住分配接口是系统业务规则最集中的接口，其实现要点为：方法标注 @Transactional 保证事务性；首先按 ID 查询床位并校验其状态必须为“空闲”，再校验该学生不存在状态为“在住”的入住记录，任一校验不通过即抛出业务异常并回滚；随后将床位置为“已入住”并绑定学生，最后生成入住日期为当天、状态为“在住”的入住记录。为避免关联对象仅有 ID 的“空壳”数据被写入持久化上下文造成返回数据不完整，实现中先按 ID 从仓库加载完整的学生实体再进行绑定。核心代码片段如下：

```
@PostMapping("/assign")
@Transactional
public Result<CheckIn> assign(@RequestBody AssignReq req) {
    Bed bed = beds.findById(req.bedId)
        .orElseThrow(() -> new BizException("床位不存在"));
    if (!"空闲".equals(bed.getStatus()))
        throw new BizException("该床位不是空闲状态，无法分配");
```

```
        if (checkIns.findByStudentIdAndStatus(req.studentId, "在
    住").isPresent())
            throw new BizException("该学生已在住，不能重复分配");
        Student stu = students.findById(req.studentId)
            .orElseThrow(() -> new BizException("学生不存在"));
        bed.setStatus("已入住"); bed.setStudent(stu); beds.save(bed);
        CheckIn ci = new CheckIn();
        ci.setStudent(stu); ci.setRoom(bed.getRoom()); ci.setBed(bed);
        ci.setCheckInDate(LocalDate.now()); ci.setStatus("在住");
        CheckIn saved = checkIns.save(ci);
        return
    Result.ok(checkIns.findById(saved.getId()).orElse(saved));
    }
```

调宿接口在同一事务中先释放旧床位（置空闲、解绑学生），再按相同规则占用新床位并更新入住记录的房间、床位与备注（追加调宿日期留痕）；退宿接口将入住记录置为“已退宿”、填写退宿日期并释放其床位。三者共同保证了床位状态、学生绑定与入住记录三处数据始终一致。

第 6 章　系统测试

6.1　测试环境与方法

测试在系统实际运行环境中进行：后端服务与 MySQL 数据库部署于同一台 Linux 服务器，前端以构建后的静态资源形式由后端提供。本章采用黑盒测试方法，依据需求分析设计测试用例，通过浏览器操作与接口调用工具相结合的方式执行测试，重点验证功能正确性、业务规则约束与权限控制的有效性。测试数据为系统初始化的演示数据（2 栋楼、24 间房、96 个床位、12 名学生、10 条在住记录等）。

6.2　功能测试

主要功能测试用例与实际执行结果如表 6-1 所示。

表 6-1　功能测试用例与结果

编号	测试内容	预期结果	实际结果
T01	管理员使用正确账号 admin/admin123 登录	登录成功，返回令牌并进入首页	通过：返回 JWT（角色 ADMIN），进入仪表盘
T02	使用错误密码登录	提示用户名或密码错误	通过：返回 code=400 及错误提示
T03	未携带令牌访问业务接口	拒绝访问	通过：返回 403
T04	学生账号访问管理员专属的账号管理接口	拒绝访问	通过：返回 403
T05	仪表盘统计数据核对	学生数 12、房间 24、床位 96、已入住 10、入住率 10.4%	通过：统计值与数据库实际数据一致
T06	为不在住学生分配空闲床位	分配成功，床位置为已入住并绑定学生，生成在住记录	通过
T07	对已占用床位或已在住学生重复分配	拒绝并提示原因，不产生数据变更	通过：返回“该床位不是空闲状态，无法分配”
T08	为学生办理调宿	旧床位释放、新床位占用、入住记录同步更新并留痕	通过
T09	为学生办理退宿	记录置为已退宿并填写退宿日期，床位恢复空闲	通过
T10	学生自助注册新账号	创建登录账号与学生	通过

		档案，可立即登录	
T11	学生登录查看“我的宿舍”	正确显示本人楼栋、房号、床号与入住日期	通过
T12	各模块分页、按关键字搜索、新增、编辑、删除	功能正常，数据变更即时生效	通过（学生、楼栋、房间、床位、报修、卫生、访客、水电费、请假、公告等模块逐一验证）
T13	管理员重置学生账号密码并禁用账号	重置后可用初始密码登录；禁用后无法登录	通过

6.3 测试结果分析

测试结果表明：系统全部核心功能均按设计实现，入住分配的双重校验能够有效阻止床位重复分配与学生重复入住等错误操作，调宿、退宿后床位状态与入住记录保持一致；JWT 认证与角色授权工作正常，未登录与越权访问均被正确拦截；仪表盘统计数据与数据库实际数据逐项核对一致。测试过程中发现并修复了入住分配接口返回数据中关联学生信息为空的问题（原因为使用仅含 ID 的占位实体进行绑定，修复为先加载完整实体）。系统整体运行稳定，达到了预期的设计目标。

第 7 章　总结与展望

7.1 总结

本文针对传统宿舍管理方式数据分散、效率低下、统计困难等问题，设计并实现了一套基于 Spring Boot + Vue 的宿舍管理系统。系统采用前后端分离架构，后端基于

Spring Boot 3、Spring Security、JWT 与 Spring Data JPA，前端基于 Vue 3、Element Plus 与 ECharts，数据库采用 MySQL 8.0。系统实现了面向管理员、宿管、学生三类角色的十余个功能模块，覆盖学生档案、楼栋房间床位、入住分配与调宿退宿、报修、卫生检查、访客登记、水电费、请假审批、通知公告、统计分析与账号管理等宿舍管理全流程业务，并对入住分配等关键业务实施了严格的规则校验与事务控制。开发过程中，通过后端通用 CRUD 控制器基类与前端配置驱动的通用 CRUD 页面组件两处复用设计，大幅减少了重复代码，提高了开发效率与系统的可维护性。经测试，系统功能完整、运行稳定，能够满足高校宿舍日常管理的信息化需求。

7.2 展望

受时间与条件所限，系统仍有可改进之处：第一，可增加查寝考勤、宿舍用电安全监测等模块，并与校园门禁、一卡通系统对接，实现数据互通；第二，可引入微信小程序或移动端适配，方便学生随时办理业务、接收公告与审批结果推送；第三，可对水电费模块扩展在线支付与自动抄表数据接入；第四，可进一步完善数据备份、操作日志审计与更细粒度的权限控制（如按楼栋划分宿管管理范围），并通过缓存、索引优化等手段提升大数据量下的查询性能。上述改进将使系统更加完善，更好地服务于智慧校园建设。

参考文献

[1] Craig Walls. Spring in Action（Spring 实战）[M]. 6 版. 北京: 人民邮电出版社, 2022.

[2] 陈木鑫. Spring Boot 企业级开发教程[M]. 2 版. 北京: 人民邮电出版社, 2021.

[3] 储久良. Vue.js 3.x 前端开发实战[M]. 北京: 清华大学出版社, 2022.

[4] 尤雨溪. Vue 3 官方文档[EB/OL]. https://cn.vuejs.org, 2024.

[5] Spring. Spring Boot 3.3 Reference Documentation[EB/OL]. https://docs.spring.io/spring-boot, 2024.

[6] 王珊, 萨师煊. 数据库系统概论[M]. 5 版. 北京: 高等教育出版社, 2014.

[7] 张海藩, 牟永敏. 软件工程导论[M]. 6 版. 北京: 清华大学出版社, 2013.

[8] 李刚. 疯狂 Java 讲义[M]. 5 版. 北京: 电子工业出版社, 2021.

[9] 周志明. 深入理解 Java 虚拟机[M]. 3 版. 北京: 机械工业出版社, 2019.

[10] 黑马程序员. MySQL 数据库原理与应用[M]. 北京: 人民邮电出版社, 2020.

[11] Jones M, Bradley J, Sakimura N. JSON Web Token (JWT): RFC 7519[S]. IETF, 2015.

[12] Fielding R T. Architectural Styles and the Design of Network-based Software Architectures[D]. Irvine: University of California, 2000.

[13] Apache ECharts. ECharts 官方文档[EB/OL]. https://echarts.apache.org, 2024.

[14] Element Plus 团队. Element Plus 组件库官方文档[EB/OL]. https://element-plus.org, 2024.

[15] 刘伟, 赵亮. 基于 Spring Boot 与 Vue 的高校宿舍管理系统设计与实现[J]. 电脑知识与技术, 2023, 19(12): 45-48.

[16] 张帆, 王静. 前后端分离架构在校园信息系统中的应用研究[J]. 信息技术与信息化, 2022(8): 112-115.

致　谢

在毕业设计完成之际，谨向给予我指导与帮助的老师、同学和家人表示衷心感谢。首先感谢指导教师在选题、系统设计与论文撰写过程中给予的悉心指导和宝贵意见；感谢同学们在技术选型与测试阶段提供的帮助与建议；感谢家人一直以来的理解与支持。由于本人水平有限，论文与系统中难免存在不足之处，恳请各位老师批评指正。

附　录

附录 A　数据库表清单

系统数据库 dorm_db 共包含 12 张数据表：sys_user（系统用户）、student（学生）、building（宿舍楼）、room（房间）、bed（床位）、check_in（入住记录）、repair（报修工单）、hygiene（卫生检查）、visitor（访客登记）、fee（水电

费）、dorm_leave（请假申请）、notice（通知公告）。完整建表语句见随附源码中的 sql/schema.sql 文件。

附录 B　系统演示账号

角色	用户名	密码	说明
系统管理员	admin	admin123	全部功能权限
宿舍管理员	manager	manager123	日常业务管理
学生	20230101（学号）	123456	演示学生陈小明，其余学生账号为各自学号、密码均为 123456